Overview

Here's a compact, formal logic grammar you can treat as a spec for reasoning about Recognition vs Comprehension as distinct cognitive states.

---

1. Symbols and signatures

Individual constants:

- Entities: $e \in \mathcal{E}$
- Agents: $a \in \mathcal{A}$
- Times: $t \in \mathcal{T}$

Predicate symbols (typed):

- $\text{Rec}(a,e,t)$: agent a is in Recognition state for entity e at time t
- $\text{Comp}(a,e,t)$: agent a is in Comprehension state for entity e at time t
- $\text{ModelExists}(a,e,t)$: a has a generative model for e at t
- $\text{Predicts}(a,e,t)$: model for e yields successful predictions at t
- $\text{Tracks}(a,e,t)$: a continuously tracks e at t
- $\text{Notices}(a,e,t)$: a consciously notices e at t
- $\text{SurpriseHigh}(a,e,t)$: surprise about e is above threshold at t
- $\text{SurpriseLow}(a,e,t)$: surprise about e is below threshold at t

You can extend with more predicates (e.g., $\text{CausalGraphIntegrated}(a,e,t)$) as needed.

---

2. Syntax (BNF-style grammar)

2.1 Terms

```bnf
<Term> ::= <Entity> | <Agent> | <Time> | <Var>

<Entity> ::= e1 | e2 | e3 | ...
<Agent>  ::= a1 | a2 | a3 | ...
<Time>   ::= t0 | t1 | t2 | ...

<Var> ::= x | y | z | a | e | t | ...
```

2.2 Atomic formulas

```bnf
<Atom> ::= "Rec(" <Term> "," <Term> "," <Term> ")"
        | "Comp(" <Term> "," <Term> "," <Term> ")"
        | "ModelExists(" <Term> "," <Term> "," <Term> ")"
        | "Predicts(" <Term> "," <Term> "," <Term> ")"
        | "Tracks(" <Term> "," <Term> "," <Term> ")"
        | "Notices(" <Term> "," <Term> "," <Term> ")"
        | "SurpriseHigh(" <Term> "," <Term> "," <Term> ")"
        | "SurpriseLow(" <Term> "," <Term> "," <Term> ")"
        | <RelationalAtom>

<RelationalAtom> ::= <PredicateSymbol> "(" <TermList> ")"

<PredicateSymbol> ::= P | Q | R | ...
<TermList> ::= <Term> | <Term> "," <TermList>
```

2.3 Well-formed formulas (WFFs)

```bnf
<Formula> ::= <Atom>
           | "¬" <Formula>
           | <Formula> "∧" <Formula>
           | <Formula> "∨" <Formula>
           | <Formula> "→" <Formula>
           | <Formula> "↔" <Formula>
           | "∀" <Var> "." <Formula>
           | "∃" <Var> "." <Formula>
```

This is standard first-order logic with a fixed vocabulary tailored to recognition/comprehension.

---

3. Core axioms (as schemas in this grammar)

You can treat these as invariant schemas expressible in the grammar above.

3.1 Mutual exclusivity (local)

```text
A1.  ∀a,e,t.  Comp(a,e,t) → ¬Rec(a,e,t)
```

A2. ∀a,e,t. Rec(a,e,t) → ¬Comp(a,e,t)
`

3.2 Comprehension requires model and prediction

`text
A3. ∀a,e,t. Comp(a,e,t) → ModelExists(a,e,t)
A4. ∀a,e,t. Comp(a,e,t) → Predicts(a,e,t)
A5. ∀a,e,t. Comp(a,e,t) → Tracks(a,e,t)
A6. ∀a,e,t. Comp(a,e,t) → SurpriseLow(a,e,t)
`

3.3 Recognition without comprehension

`text
A7. ∀a,e,t. Rec(a,e,t) → ¬ModelExists(a,e,t) ∨ ¬Predicts(a,e,t)
A8. ∀a,e,t. Rec(a,e,t) → SurpriseHigh(a,e,t) ∨ ¬Tracks(a,e,t)
`

---

4. Noticing as evidence of incomplete comprehension

Your key invariant, in-grammar:

`text
A9. ∀a,e,t. Notices(a,e,t) → SurpriseHigh(a,e,t)
A10. ∀a,e,t. Notices(a,e,t) → ¬Comp(a,e,t⁻)
`

Where \(t^- \) is “immediately before \(t\)”. If you want to stay strictly first-order, you can encode this with a predecessor relation:

`text
Pred(t', t) ≡ "t' is the immediate predecessor of t"

A10'. ∀a,e,t,t'. (Pred(t',t) ∧ Notices(a,e,t)) → ¬Comp(a,e,t')
`

---

5. Transition rules (expressed as implications)

5.1 Recognition → Comprehension (upgrade)

`text
T1. ∀a,e,t. (Rec(a,e,t) ∧ ModelExists(a,e,t) ∧ Predicts(a,e,t) ∧ Tracks(a,e,t) ∧ SurpriseLow(a,e,t))
        → Comp(a,e,t)
`

5.2 Comprehension → Recognition (degradation)

`text
T2. ∀a,e,t. (Comp(a,e,t) ∧ SurpriseHigh(a,e,t)) → Rec(a,e,t)
`

(You can refine T2 with additional predicates like ModelDrift, Overload, etc.)

---

6. Example well-formed formulas

Example 1 — “If I only noticed it now, I wasn’t comprehending it just before”:

`text
∀a,e,t,t'. (Pred(t',t) ∧ Notices(a,e,t)) → ¬Comp(a,e,t')
`

Example 2 — “Comprehension implies predictive tracking with low surprise”:

`text
∀a,e,t. Comp(a,e,t) → (Predicts(a,e,t) ∧ Tracks(a,e,t) ∧ SurpriseLow(a,e,t))
`